



Data Structures

Part 1: Lists

Storing groups of data together in a single variable.

Lists

List: A data structure that stores multiple values in a specific order within a single variable.

Lists are useful when working with large amounts of similar or related data.

Lists are iterables, meaning they can easily be iterated over.

Iterating over a list allows you to perform some kind of computation using every element of the list, with relatively little code.



Creating Lists

Lists are created in Python using the square bracket `[]` symbols.

A list can be created by setting a variable equal to a sequence of values contained in a pair of square brackets `[]`.

Values are separated by commas `(,)`.

Any data can be added to a list, including a mix of different datatypes.

```
# A list of ints
my_numbers = [0, 1, 2, 3, 4, 5]

# A list of strings
my_strings = ["this", "is", "a", "list"]

# A mixed list
my_list = [7, "hello world", 10.5, True]
```

Creating Lists *continued...*

Alternatively, you can also create lists with items on their own line.

This can make code easier to read and debug later, depending on the length and contents of the list.

```
shopping_list = [  
    "eggs",  
    "milk",  
    "cheese",  
    "bread",  
    "bananas"  
]
```

Accessing List Values

To access a value of a list, you need to use its index.

In virtually every programming language, indices start counting from **0** (this is to save on memory).

You can access a value from a list by using the following syntax:

```
list_name[index]
```

```
shopping_list = ["eggs", "milk", "bread"]  
  
# Print the first value  
print(shopping_list[0])  
# Print the second value  
print(shopping_list[1])  
# Print the second value  
print(shopping_list[2])
```

OUTPUT

```
eggs  
milk  
bread
```

Accessing List Values *continued...*

If you try and access an index out of range, you will get an error.

Python does not allow you to access indices that are out of range.

```
shopping_list = [  
    "eggs",  
    "milk",  
    "bread"  
]  
# Index 3 is out of range  
print(shopping_list[3])
```

OUTPUT

```
IndexError: list index out of range
```

Accessing List Values *continued...*

Accessing an index can also be used to reassign the value located at that index.

When you assign a value to an index, the value in the list at that index changes to the newly assigned value.

```
shopping_list = [  
    "eggs",  
    "milk",  
    "bread"  
]  
  
shopping_list[1] = "cheese"  
  
print(shopping_list)
```

```
OUTPUT  
['eggs', 'cheese', 'bread']
```

Negative Indices

Negative indices let you access items starting from the end of a list instead of the beginning.

An index of **-1** accesses the last item, **-2** accesses the second-last item, and so on.

This is useful when you need to access items near the end without knowing the length of the list.

```
colors = ["red", "green", "blue", "yellow"]  
  
print(colors[-1])  
print(colors[-2])
```

OUTPUT

```
yellow  
blue
```

Adding Items to Lists – append()

`append()` adds a single item to the end of a list.

It is useful when you are building a list and want to add new values one at a time.

```
colors = ["red", "green"]
```

```
colors.append("blue")
```

```
print(colors)
```

OUTPUT

```
["red", "green", "blue"]
```

Adding Items to Lists – `insert()`

`insert()` adds an item at a specific index in a list.

Items at and after that index are shifted back to make room for the new item.

```
shopping_list = [  
    "apples",  
    "bananas",  
    "milk",  
    "oranges"  
]  
  
shopping_list.insert(0, "bread")  
  
print(shopping_list)
```

```
OUTPUT  
["bread", "apples", "bananas", "milk", "oranges"]
```

Adding Items to Lists – extend()

`extend()` adds all the items from another collection to the end of a list.

It is useful for combining the contents of two lists without creating a nested list.

```
colors = ["red", "green"]
more_colors = ["blue", "yellow"]

colors.extend(more_colors)

print(colors)
```

OUTPUT

```
["red", "green", "blue", "yellow"]
```

Removing Items from Lists – `remove()`

`remove()` searches for a value and removes the first matching item from the list.

It is useful when you know the value you want to remove rather than its index.

```
colors = ["red", "green", "blue"]  
  
colors.remove("green")  
  
print(colors)
```

OUTPUT

```
["red", "blue"]
```

Removing Items from Lists – pop()

pop() removes an item from a list and returns the removed value.

By default, it removes the last item, but you can provide an index to remove an item from another position.

```
shopping_list = ["apples", "bananas", "milk", "oranges"]  
  
last_item = shopping_list.pop()  
  
first_item = shopping_list.pop(0)  
  
print(shopping_list)  
print(last_item)  
print(first_item)
```

OUTPUT

```
["bananas", "milk"]  
oranges  
apples
```

Removing Items from Lists – `clear()`

`clear()` removes every item from a list, leaving the list empty.

It is useful when you want to reset a list while continuing to use the same list variable.

```
colors = ["red", "green", "blue"]  
  
colors.clear()  
  
print(colors)
```

OUTPUT

```
[]
```

Empty Lists

An empty list is a list that starts with no items and can have values added to it later.

Empty lists are useful when you do not know what values the list will contain when the program begins.

For example, a program might start with an empty list and add values as they are entered, calculated, or discovered while the program runs.

```
my_list = []
```

List Length with `len()`

`len()` returns the number of items stored in a list.

It is useful when you need to know how many values are in the list, such as when displaying a count or controlling a loop.

The value returned by `len()` is an integer.

```
colors = [  
    "red",  
    "green",  
    "blue"  
]  
  
print(len(colors))
```

OUTPUT

3

Checking Membership with `in`

The `in` keyword checks whether a specific value exists within a list.

It returns `True` if the value is found and `False` if it is not.

This is especially useful with conditional statements when a program needs to behave differently depending on whether a value is in a list.

```
colors = ["red", "green", "blue"]
```

```
if "green" in colors:  
    print("Green is in the list!")
```

OUTPUT

```
Green is in the list!
```

Traversing Lists with for

A **for** loop can traverse a list by accessing each value in the list one at a time.

The loop variable temporarily stores the current value from the list during each iteration.

The variable can be used inside the loop to work with that value, and it automatically changes to the next value as the loop continues.

The loop ends after every value in the list has been accessed.

```
colors = ["red", "green", "blue"]  
  
for color in colors:  
    print(color)
```

```
OUTPUT  
red  
green  
blue
```

Traversing Lists with `for` *continued...*

In this example, we see that you can use a `for` loop to perform a computation on the data for each iteration of the loop.

```
numbers = [4, 7, 10, 13, 16]

for number in numbers:
    if number >= 10:
        print(number)
    else:
        print("Less than 10.")
```

OUTPUT

```
Less than 10.
Less than 10.
10
13
16
```

Copying Lists

The `copy()` method creates a new list containing the same values as the original list.

Because the copied list is a separate list, adding, removing, or replacing items in it does not change the original list.

This is useful when you want to modify a list while keeping the original version unchanged.

```
original_colors = ["red", "green", "blue"]
copied_colors = original_colors.copy()

copied_colors.append("yellow")

print(original_colors)
print(copied_colors)
```

OUTPUT

```
["red", "green", "blue"]
["red", "green", "blue", "yellow"]
```

Copy vs Reference

When you assign an existing list to another variable, Python does not create a new list.

Instead, the new variable becomes a reference to the same list, meaning both variables point to the same data in memory.

As a result, changing the list through either variable changes the same list.

To create a separate list that can be changed independently, you can use the `copy()` method.

```
original = ["red", "green", "blue"]

reference = original
copied = original.copy()

reference.append("yellow")
copied.append("purple")

print(f"Original: {original}")
print(f"Reference: {reference}")
print(f"Copy: {copied}")
```

OUTPUT

```
Original: ['red', 'green', 'blue', 'yellow']
Reference: ['red', 'green', 'blue', 'yellow']
Copy: ['red', 'green', 'blue', 'purple']
```